

MI DPU API

Version 0.1

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
0.1	Initial release	01/10/2022

TABLE OF CONTENTS

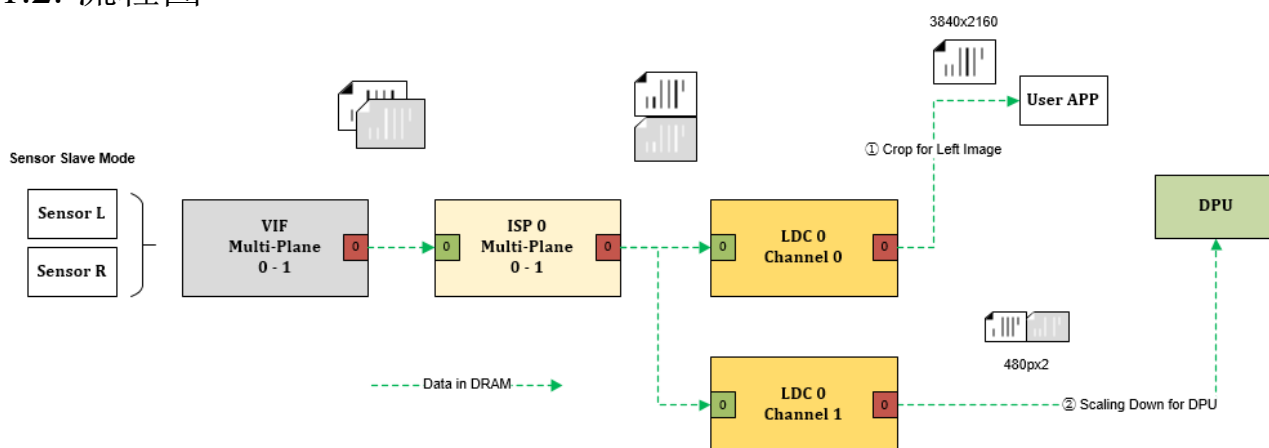
REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. API 参考.....	1
1.1. 概述.....	1
1.2. 流程图.....	1
1.3. 关键说明.....	1
2. 功能模块 API.....	3
2.1.1 MI_DPU_CreateDevice.....	3
2.1.2 MI_DPU_DestroyDevice.....	4
2.1.3 MI_DPU_CreateChannel.....	5
2.1.4 MI_DPU_DestroyChannel.....	6
2.1.5 MI_DPU_SetChnParam.....	6
2.1.6 MI_DPU_GetChnParam.....	7
2.1.7 MI_DPU_StartChannel.....	8
2.1.8 MI_DPU_StopChannel.....	9
3. 数据类型.....	10
3.1. 数据结构列表.....	10
3.2. MI_DPU_DEV.....	10
3.3. MI_DPU_CAHNNEL.....	10
3.4. MI_DPU_DevAttr_t.....	11
3.9. MI_DPU_ChnAttr_t.....	13
3.10. MI_DPU_ChnParam_t.....	14
4. 错误码.....	15

1. API 参考

1.1. 概述

DPU 通过对视频双摄像头输入的左右两张图像，进行特定算法处理后，输出与输入图像对应的深度信息。通过计算可获得图像中物体相对摄像头的距离。

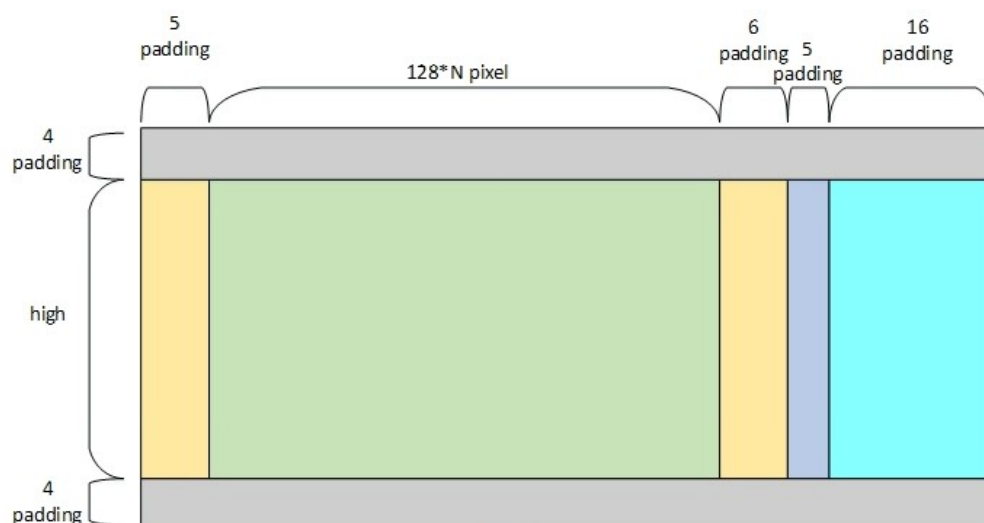
1.2. 流程图



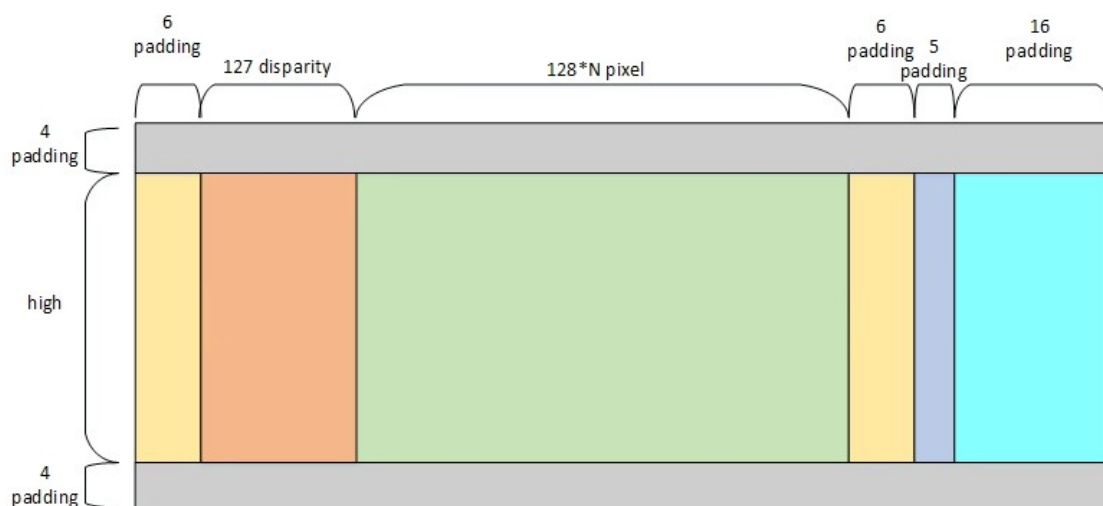
1.3. 关键说明

DPU 输入:

左图:



右图:



上面两张图分别表示给到 DPU 的左图和右图 buffer。绿色部分为 sensor 实际拍摄的内容，宽度要求为 128pixel 的整数倍。其余颜色部分皆为 padding 内容。

1.左图黄色 padding=5 的宽度是算法要求，padding 内容是绿色区域第一列内容。右边黄色 padding=6 宽度是算法要求，蓝紫色 padding=5 是为了 16pixel 对齐，天蓝色部分是算法要求，固定 padding=16。padding 内容都为绿色区域右边一列。

2.右图大致和左图要求一致，只是因为多出了橙色 padding=127pixel 的宽度（Muffin 因为目前只支持 disparity=64，故 padding=127 须改为 padding=63），为了做 128 对齐，黄色 padding=5+1=6，黄色和橙色 padding 内容一样，都是绿色区域第一列内容。右边黄色 padding=6 宽度是算法要求，蓝紫色 padding=5 是为了 16pixel 对齐，天蓝色部分是算法要求，固定 padding=16。padding 内容都为绿色区域右边一列。

上图 buffer 由 DPU 的前置模块 LDC 输出，我们会提供 cv tool 用来制作供 LDC 使用的 mapping table。cv tool 的使用方法请参考文档：《CV Tool 使用说明》

Device:

DPU 硬件设备

Channel:

DPU 设备的通道，各个通道分时复用 DPU 硬件。（目前仅支持 channel 0）

输入图像格式:

YUV420

输出图像格式:

disparity – 视差

输入分辨率:

Muffin:

仅支持 640 x 480

输出分辨率:

与输入分辨率一致

2. 功能模块 API

如下是 MI DPU 模块的用户接口。

API 名	功能
系统功能类	
MI_DPU_CreateDevice	创建 DPU 设备
MI_DPU_DestroyDevice	销毁 DPU 设备
MI_DPU_CreateChannel	创建 DPU 通道
MI_DPU_DestroyChannel	销毁 DPU 通道
MI_DPU_StartChannel	启用 DPU 通道
MI_DPU_StopChannel	禁用 DPU 通道
MI_DPU_SetChnParam	配置 DPU 通道属性
MI_DPU_GetChnParam	获取 DPU 通道属性

2.1.1 MI_DPU_CreateDevice

➤ 功能

创建 DPU 设备。

➤ 语法

MI_S32 MI_DPU_CreateDevice(MI_DPU_DEV DevId , [MI_DPU_DevAttr_t](#) *pstDevAttr);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID, 对应 DPU Device ID	输入
pstDevAttr	Device 设备属性, 静态参数	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败, 参照[错误码](#)。

➤ 依赖

- 头文件: mi_dpu.h
- 库文件: libmi_dpu_user.a / libmi_dpu_user.so

※ 注意

- 目前只支持 DevId =0;

➤ 举例

DPU 初始化设置, 以及退出范例

```
MI_DPU_DEV DevId =0;
```

```
MI_DPU_CHANNEL ChnId =0;
MI_DPU_DevAttr_t stDevAttr = {0};
MI_DPU_ChnAttr_t stChnAttr = {0};

stChnAttr.u16Width = 640;//设置 CHN window 宽度
stChnAttr.u16Height = 480;//设置 CHN window 高度
stChnAttr.u16MinDisparity = 0;//设置 DPU 起始 disparity
stChnAttr.u6MaxDisparity = 64;//设置 DPU 最大 disparity

MI_DPU_CreateDevice(DevId, &stDevAttr);
MI_DPU_CreateChannel(DevId, ChnId, &stChnAttr);
MI_DPU_StartChannel(DevId, ChnId);

/*销毁通道*/
MI_DPU_StopChannel(DevId, ChnId);
MI_DPU_DestroyChannel(DevId, ChnId);
MI_DPU_DestroyDevice(DevId)
```

- 相关的主题
MI_DPU_DestroyDevice

2.1.2 MI_DPU_DestroyDevice

- 功能
销毁 DPU 设备。
- 语法
MI_S32 MI_DPU_DestroyDevice(MI_DPU_DEV DevId);

- 形参

参数名称	描述	输入/输出
DevId	Device ID	输入

- 返回值
 - MI_SUCCESS(0) 成功。
 - 非 0 失败，参照[错误码](#)。
- 依赖
 - 头文件：mi_dpu.h
 - 库文件：libmi_dpu_user.a / libmi_dpu_user.so
- ※ 注意
 - 目前只支持 DevId =0;

- 举例
见 MI_DPU_CreateDevice 举例

- 相关的主题
MI_DPU_CreateDevice

2.1.3 MI_DPU_CreateChannel

- 功能
创建 DPU Channel 通道。

- 语法
MI_S32 MI_DPU_CreateChannel(MI_DPU_DEV DevId , MI_DPU_CHANNEL ChnId, [MI_DPU_ChnAttr_t](#) *pstChAttr);

- 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入
pstChAttr	Channel 属性	输入

- 返回值
 - MI_SUCCESS(0) 成功。
 - 非 0 失败，参照[错误码](#)。
- 依赖
 - 头文件：mi_dpu.h
 - 库文件：libmi_dpu_user.a / libmi_dpu_user.so

- ※ 注意
 - 目前只支持 ChnId =0;
 - 需要在 MI_DPU_CreateDevice 之后调用

- 举例
见 MI_DPU_CreateDevice 举例

- 相关的主题
MI_DPU_DestroyChannel

2.1.4 MI_DPU_DestroyChannel

- 功能
销毁 DPU Channel 通道。

➤ 语法

MI_S32 MI_DPU_DestroyChannel(MI_DPU_DEV DevId , MI_DPU_CHANNEL ChnId);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_dpu.h
- 库文件：libmi_dpu_user.a / libmi_dpu_user.so

※ 注意

- 目前只支持 ChnId =0;
- 需要在 MI_DPU_CreateDevice 之后调用

➤ 举例

见 MI_DPU_CreateDevice 举例

➤ 相关的主题

MI_DPU_CreateChannel

2.1.5 MI_DPU_SetChnParam

➤ 功能

设置 DPU Channel 动态参数

➤ 语法

MI_S32 MI_DPU_SetChnParam(MI_DPU_DEV DevId , MI_DPU_CHANNEL ChnId, MI_DPU_ChnParam_t *pstChnParam);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入
pstChnParam	Channel 动态参数	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

- 依赖
 - 头文件: mi_dpu.h
 - 库文件: libmi_dpu_user.a / libmi_dpu_user.so

- ※ 注意
 - 目前只支持 ChnId =0;
 - Device 和 channel 创建之后调用

- 举例

```
MI_DPU_DEV  DevId =0;
MI_DPU_CHANNEL  ChnId =0;

MI_DPU_ChnParam_t stChnParam = {0};

MI_DPU_GetChnParam(DevId , ChnId,& stChnParam);

stChnParam.u16Width = 640;//设置 CHN window 宽度
stChnParam.u16Height = 480;//设置 CHN window 高度
stChnParam.u16MinDisparity = 0;//设置 DPU 起始 disparity
stChnParam .u6MaxDisparity = 64;//设置 DPU 最大 disparity

MI_DPU_SetChnParam(DevId , ChnId, & stChnParam);
```

- 相关的主题
 - MI_DPU_GetChnParam

2.1.6 MI_DPU_GetChnParam

- 功能
 - 获取 DPU Channel 动态参数

- 语法

MI_S32 MI_DPU_GetChnParam(MI_DPU_DEV DevId , MI_DPU_CHANNEL ChnId, MI_DPU_ChnParam *pstChnParam);

- 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入
pstChnParam	Channel 动态参数	输出

- 返回值
 - MI_SUCCESS(0) 成功。
 - 非 0 失败, 参照[错误码](#)。
- 依赖

- 头文件: mi_dpu.h
- 库文件: libmi_dpu_user.a / libmi_dpu_user.so

※ 注意
N/A

➤ 举例
见 MI_DPU_SetChnParam 举例

➤ 相关的主题
MI_DPU_SetChnParam

2.1.7 MI_DPU_StartChannel

➤ 功能
启动 DPU Channel。

➤ 语法
MI_S32 MI_DPU_StartChannel(MI_DPU_DEV DevId , MI_DPU_CHANNEL ChnId);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件: mi_dpu.h
- 库文件: libmi_dpu_user.a / libmi_dpu_user.so

※ 注意

- 目前只支持 ChnId =0;
- Device 和 channel 创建之后调用

➤ 举例
见 MI_DPU_CreateDevice 举例

➤ 相关的主题
MI_DPU_StopChannel

2.1.8 MI_DPU_StopChannel

➤ 功能
禁用 DPU Channel 通道。

➤ 语法

MI_S32 MI_DPU_StopChannel(MI_DPU_DEV DevId , MI_DPU_CHANNEL ChnId);

➤ 形参

参数名称	描述	输入/输出
DevId	Device ID	输入
ChnId	Channel ID	输入

➤ 返回值

- MI_SUCCESS(0) 成功。
- 非 0 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_dpu.h
- 库文件：libmi_dpu_user.a / libmi_dpu_user.so

※ 注意

N/A

➤ 举例

见 MI_DPU_CreateDevice 举例

➤ 相关的主题

MI_DPU_StartChannel

3. 数据类型

3.1. 数据结构列表

相关数据类型、数据结构定义如下：

MI_DPU_DEV	定义 DPU Device 的类型
MI_DPU_CHANNEL	定义 DPU Channel 的类型
MI_DPU_DevAttr_t	定义 DPU Device 属性
MI_DPU_ChnAttr_t	定义 DPU Channel 的属性。
MI_DPU_ChnParam_t	定义 DPU Channel 的动态参数

3.2. MI_DPU_DEV

- 说明
定义 DPU Device 的类型。
- 定义
`typedef MI_U32 MI_DPU_DEV`
- ※ 注意事项
N/A
- 相关数据类型及接口
N/A

3.3. MI_DPU_CHANNEL

- 说明
定义 DPU channel 的类型。
- 定义
`typedef MI_U32 MI_DPU_CHANNEL`
- ※ 注意事项
N/A
- 相关数据类型及接口
N/A

3.4. MI_DPU_DevAttr_t

- 说明
定义设备 Device 的属性。
- 定义

```
typedef struct MI_DPU_DevAttr_s
{
    MI_U32 u32Reserved;
} MI_DPU_DevAttr_t;
```

- 成员

成员名称	描述
u32Reserved	保留数据

- ※ 注意事项
N/A
- 相关数据类型及接口
N/A

3.5. MI_DPU_ChnAttr_t

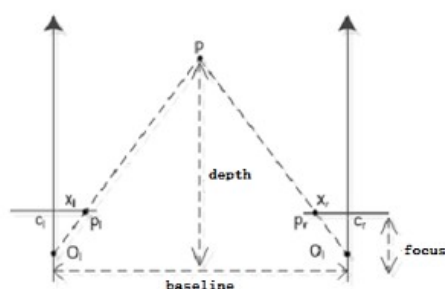
➤ 说明

定义 DPU Channel 的属性。创建 Channel 时需设置。

如何计算 MinDisparity 和 MaxDisparity:

基线，像素焦距和视差与深度的关系式如下：

$$\frac{focus_{pixel} * baseline}{disparity} = depth$$



假设我们需要覆盖的工作范围为：0.6~4.1m

假定 focus = 900 pixel, baseline = 0.1m, 则

maxDisparity = 900 * 0.1 / 0.6 = 150;

minDisparity = 900 * 0.1 / 4.1 = 21.95;

或

minDisparity = maxDisparity - disparity = 150 - 128 = 22;

maxDisparity 决定了最近物距的下限，minDisparity 决定了最远物距的上限。物距越近，则工作范围越小。

反之则越大。故在焦距，基线和分辨率确定的情况下，工作范围有以下几档：

focus = 900pixel↵
resolution = 1920*1080↵
baseline = 10cm↵
disparity size = 128m↵
Zmin=0.1m, Zmax=0.116m↵
Zmin=0.2m, Zmax=0.28m↵
Zmin=0.3m, Zmax=0.52m↵
Zmin=0.5m, Zmax=1.7m↵
Zmin=0.6m, Zmax=4.1m↵
Zmin>0.7m, Zmax=90m↵

注：Muffin 目前只支持 disparity=64，同时分辨率只支持 640 x 480。假定送数据给到 DPU 前将 1920 x 1080 缩放到 640 x 480，对应的像素焦距也要等比例缩小 3 倍，即 $focus_{pixel} = 900 / 3 = 300$ （镜头实际焦距还是 900pixel，只是用户在计算 minDisparity 和 maxDisparity 时需要做等比例缩小）；则

focus = 300pixel↵
resolution = 640*480↵
baseline = 10cm↵
disparity size = 64m↵
Zmin=0.1m, Zmax=0.127m↵
Zmin=0.2m, Zmax=0.349m↵
Zmin=0.3m, Zmax=0.833m↵
Zmin>0.46m, Zmax=30m↵

像素焦距和毫米焦距的换算公式如下：

$$focus_{pixel} = \frac{focus_{mm}}{df}$$

其中， df 为一个像素的大小。像素的大小可以根据 CMOS 类型和分辨率估计。比如，1/2.8 英寸，分辨率为：2048 x 1536 的 CMOS，其中 1/2.8 英寸指的是成像面长方形外接圆周长的一半。

我们可以换算成像面长度约为 4mm x 3mm，则

$$df = \frac{width_{mm}}{width_{pixel}} \approx 2 * 10^{-3} mm$$

如果毫米焦距为 3.6mm，则

$$focus_{pixel} = \frac{focus_{mm}}{df} = \frac{3.6}{2 * 10^{-3}} = 1800 pixel$$

➤ 定义

```
typedef struct MI_DPU_ChnAttr_s
{
    MI_U16 u16Width;
    MI_U16 u16Height;
    MI_U16 u16MinDisparity;
    MI_U16 u16MaxDisparity;
} MI_DPU_ChnAttr_t;
```

➤ 成员

成员名称	描述
u16Width	input frame 宽度
u16Height	input frame 高度
u16MinDisparity	最小视差
u16MaxDisparity	最大视差

※ 注意事项

N/A

➤ 相关数据类型及接口

N/A

3.6. MI_DPU_ChnParam_t

- 说明
定义 DPU Channel 的动态参数。

- 定义
typedef struct MI_DPU_ChnParam_s
{
 MI_U16 u16Width;
 MI_U16 u16Height;
 MI_U16 u16MinDisparity;
 MI_U16 u16MaxDisparity;
} MI_DPU_ChnParam_t;

- 成员

成员名称	描述
u16Width	input frame 宽度
u16Height	input frame 高度
u16MinDisparity	最小视差
u16MaxDisparity	最大视差

- ※ 注意事项
N/A

- 相关数据类型及接口
N/A

4. 错误码

MI DPU 模块 错误码如下表所示

错误码	宏定义	描述
0	MI_DPU_RET_SUCCESS	成功
0xA0282001	MI_ERR_DPU_NOT_INIT	模块未完成初始化
0xA0282002	MI_ERR_DPU_DEV_INVALID	Device 超出合法范围
0xA0282003	MI_ERR_DPU_CHN_INVALID	Channel 超出合法范围
0xA0282004	MI_ERR_DPU_TIMEOUT	DPU 设备运行超时
0xA0282005	MI_ERR_DPU_NOMEM	分配内存失败
0xA0282006	MI_ERR_DPU_NOT_SUPPORT	操作不支持
0xA0282007	MI_ERR_DPU_BUSY	Channel 系统忙
0xA0282008	MI_ERR_DPU_NOT_READY	设备未完成初始化
0xA0282009	MI_ERR_DPU_NULL_PTR	输入参数空指针错误

